Measuring epiplexity beyond transformers

A fair, small-scale comparison of MLP, CNN and transformer observers

BMM project: `episynth`

September 2026

Project question and scope

Question. Does the amount of structure found in a dataset depend on the computational observer that measures it?

- Compare an MLP and a locality-aware 1-D CNN on elementary cellular automata (ECA).
- Keep a small DiT as a transformer reference; it is a baseline, not the main contribution.
- Validate two learning-based codes and Bayesian/information-criterion baselines.

Deliverable: for each task and compute bound, report a model part (structure), a data part (unpredictable information), uncertainty over seeds, and dependence on the observer.

Two classical ways to describe information

Shannon entropy: uncertainty before the sample

$$H(X) = \mathbb{E}[-\log_2 p(X)].$$

Average bits needed to encode a draw from a known distribution.

Random bits have high entropy; a fixed alternating string has none.

Kolmogorov complexity: a recipe for one object

$$K(x) = \min_{p:\, U(p)=x} |p|.$$

Here U is a fixed universal computer. Changing U alters K only by a constant.

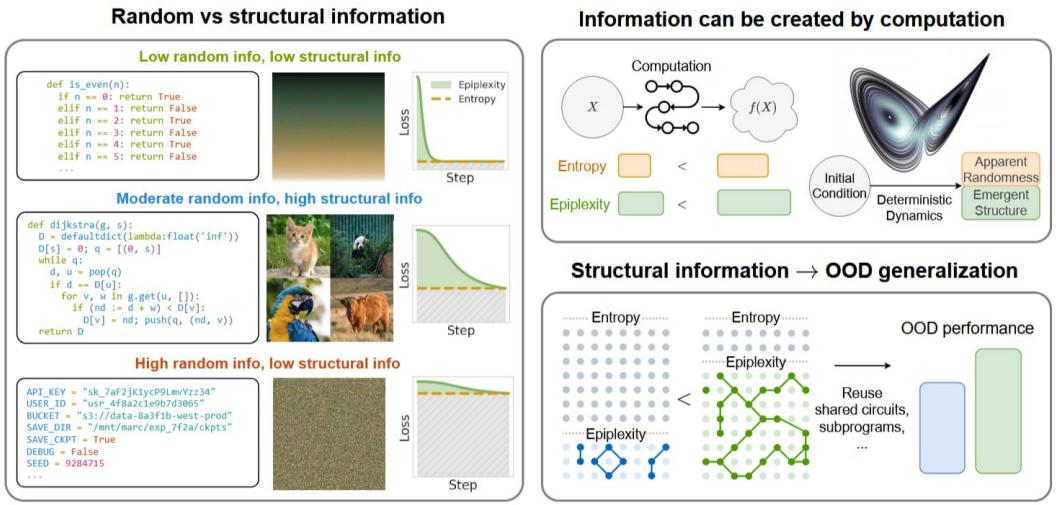
An alternating string has a short recipe; a typical random string does not. Exact $K(x)$ is uncomputable in general.

Neither notion bounds the computation used to find or apply a description.

Why introduce a third quantity?

Similar entropy does not imply similar learnability.

- One dataset contains reusable patterns.
- Another looks random to a limited learner.
- Epiplexity measures structure extracted within T steps (Finzi et al., 2026).



A learner can turn computation into reusable structure.

Definition: epiplexity

All logarithms below are base 2. Let $X \in \{0,1\}^n$ and let $\mathcal{P}_T$ be the prefix-free programs that both evaluate and sample a distribution within time $T(n)$.

For $P \in \mathcal{P}_T$, write $K(P)$ for its program length and $L_X(P) = \mathbb{E}[-\log P(X)]$. Define

$$P^* \in \arg\min_{P \in \mathcal{P}_T} \{K(P) + L_X(P)\}.$$

$$S_T(X) := K(P^*), \qquad H_T(X) := L_X(P^*).$$

Then $\mathrm{MDL}_T(X) = S_T(X) + H_T(X)$ is the total two-part description length.

Think of the code as: first send a short program describing a useful model, then send the remaining unpredictable data using that model. Epiplexity is the first part; time-bounded entropy is the second.

- S_T is the model/structure part; H_T is the residual/random part.
- Only MDL_T is guaranteed to be non-increasing as T grows; S_T need not be.

Our tasks are conditional

For supervised data, inputs are available to the decoder. With X known and labels Y to encode, the target is the conditional quantity

$$S_T(Y \mid X) = K(P^*),$$

$$P^* \in \arg\min_{P \in \mathcal{P}_T} \{K(P) + \mathbb{E}[-\log P(Y \mid X)]\}.$$

For a classifier with C classes, the per-example loss is

$$\ell_i(x, y) = -\log_2 P_i(y \mid x) \quad \text{bits per label.}$$

The input tokens count toward compute, but not toward the reported label code length.

Important distinction. The theoretical minimum ranges over all T-time programs. Our experiment restricts it to models reached by a stated architecture, optimiser and schedule; we therefore write $\widehat{S}_T^a$ and call it an estimator.

Estimator 1: prequential area

Train P_i on the first i examples of an i.i.d. stream (x_i, y_i).

Exact online code:

$$K_{\text{raw}} = \sum_{i=0}^{M-1} -\log_2 P_i(y_i \mid x_i).$$

After learning, this still pays the data entropy. To isolate the model part, estimate a floor $\widehat{L}_{\text{floor}}$ on independent held-out labels (or use the first-pass/EDL floor):

$$\widehat{S}_{\text{preq}} = \max\Big(0,\ K_{\text{raw}} - M\widehat{L}_{\text{floor}}\Big)\,.$$

The subtraction is a structure proxy, not a universal code. We report K_{raw}, $\widehat{L}_{\text{floor}}$ and $\widehat{S}_{\text{preq}}$ separately. A training-set floor is excluded: it turns memorised noise into false structure.

Estimator 2: requential coding

The encoder has a teacher P_i^{t} trained on real labels; the decoder has a student P_i^{s} trained only on teacher samples. They share initialisation, update rule, seed and batch size.

For a batch whose inputs x_b are known, the exact categorical divergence is

$$\mathrm{KL}_i = \sum_b \sum_{c=1}^{C} P_i^{\mathrm{t}}(c \mid x_b) \log_2 \frac{P_i^{\mathrm{t}}(c \mid x_b)}{P_i^{\mathrm{s}}(c \mid x_b)}.$$

Relative-entropy coding gives the expected bound

$$K_{\mathrm{req}} \leq \sum_i \left[\mathrm{KL}_i + 2\log_2(1 + \mathrm{KL}_i) + \kappa\right], \qquad \kappa < 5.21.$$

(Qiu et al., 2026)

Report both $\sum_i \mathrm{KL}_i$ and the full bound. The per-message terms matter when batches are small; the classifier extension is an experiment, not a theorem that it equals S_T.

Bayesian and classical baselines

These baselines test whether the ranking is specific to time-bounded learning codes.

Method	Quantity reported
Variational MDL	$\underbrace{\mathrm{KL}(q\|\|p)}_{\text{model part}} + \underbrace{\mathbb{E}_q[-\log p(D \mid w)]}_{\text{data part}}$; both in bits.
Laplace evidence	log-evidence/Occam term around a fitted posterior; report its model part.
EDL	$\mathbb{E}[K_{\mathrm{raw}}] - ML(P_{\mathrm{final}})$ for one fixed training algorithm (Donoway et al., 2026).
AIC, BIC, WAIC, WBIC	AIC: $-2\log L + 2k$; BIC: $-2\log L + k\log M$. WAIC/WBIC use predictive or temperature corrections.

All methods use the same task split, label units and held-out data. No raw code length is compared across methods without stating its floor and overhead.

Benchmark: ECA plus controlled data

ECA. A periodic row $x \in \{0,1\}^n$ is updated by a local rule F_r; $Y = F_r^t(X)$ with $X \sim \mathrm{Unif}(\{0,1\}^n)$. We measure $S_T(Y \mid X)$ in label bits.

Control	What it tests
Rules $15, 30, 54, 110$	contrasting learnability at fixed input size.
Constant labels	zero structural difficulty.
Shuffled or independent labels	entropy/noise floor; structure should be near zero.
Symmetric label noise ρ	$H(Y \mid X) = h(\rho) + \rho \log_2(C-1)$ per example.
Fixed pixel/input permutation	preserves MLP distribution; disrupts CNN locality.

The published transformer result gives the reference ordering rule 54 $>$ rule 15 $>$ rule 30; we test whether that ordering survives a different observer family (Finzi et al., 2026).

Observers and fair comparisons

	MLP	Circular 1-D CNN	DiT reference
Inductive bias	none	local, translation equivariant	attention + denoising
Output	conditional P	conditional P	conditional P
Primary sweep	width $\times$ depth	channels $\times$ depth	size $\times$ steps
Manipulation	example order	input permutation	sampler + steps

Within a comparison we fix the dataset, split, optimiser family, evaluation set, and compute bound. The CNN uses circular padding for ECA; otherwise locality would be confounded with boundary effects.

Protocol: from runs to $\widehat{S}_T$

1. Fix tasks, label units and a held-out set. Use at least five independent seeds.
2. Sweep widths/depths, data passes and learning rates for every observer. Log parameter count N, examples D, evaluation examples D_X, losses and wall-clock/FLOPs.
3. Use the dense-model accounting $\widetilde{C} = 6ND + 2ND_X$ only as an approximation; for CNN/DiT also record measured FLOPs. Keep points with $C_a \leq T$.
4. Plot $(C_a,\ K + M\widehat{L}_{\text{floor}})$ and take the lower convex hull. The model coordinate of the hull at T is $\widehat{S}_T^a$.
5. Repeat at several T values; never report one compute point as a curve.

Accounting rule: all losses are converted from natural logs to bits by dividing by $\ln 2$; all totals state whether they count labels only or inputs plus labels.

Validation and planned claims

For task d, observer a, bound T and seed s, store

$$(\widehat{S}^a_{T,s},\ \widehat{H}^a_{T,s},\ C_{a,s},\ \text{loss curve}).$$

- **Agreement:** Spearman ρ and Kendall τ between estimator rankings.
- **Observer effect:** $\Delta_{ab}(d,T) = \mathrm{median}_s(\widehat{S}^a_{T,s} - \widehat{S}^b_{T,s})$ with a paired bootstrap interval.
- **Seed stability:** standard deviation and probability that a ranking is preserved.
- **Controls:** random/constant labels, input permutation, and known-noise entropy floor.

Claim we can support: an estimator is observer-stable on a testbed if task orderings agree within uncertainty across setups. We will not call a setup-dependent proxy the architecture-independent theoretical S_T.

Project library: `episynth`

Package	Responsibility
`episynth.generators`	Seeded synthetic time-series generators and mixtures
`episynth.model`	Moirai observer and forecast distributions
`episynth.estimators`	Pluggable protocol; prequential implemented, requential planned
`episynth.evaluation`	GIFT-Eval windows, forecasters and metrics
`episynth.analysis`	Held-out-floor area, uncertainty and quality comparisons
`episynth.train`	JSON configurations, overrides and reproducible runs

Stack: Python, PyTorch, NumPy/SciPy, pytest, Docker and one Colab GPU. Every run is one JSON line; figures are regenerated only from saved runs.

Demo: one common result object

```
from episynth.estimators import build

# model, generator, spec and config are set by the demo.
estimator = build("prequential")
result = estimator.run(model, generator, spec, config, "runs/demo'
print(result["model_part_bits"], result["data_part_bits"])
```

The CPU notebook uses a tiny Moirai model, waveform data and 64 training windows. It plots the learning curve and the decomposition in bits. Requential remains a planned implementation; large time-series runs are stored separately from this demo.

Roles and milestones

Owner	Responsibility	BMM output
Member 1	prequential code, compute sweep and MLP/CNN ECA frontier	estimator + hull
Member 2	requential classifier code, noise/permutation tests and demo	estimator + Co-lab
Member 3	variational/Laplace codes, theory and manuscript integration	Bayesian baselines
Member 4	generators, criteria, uncertainty and packaging	reproducible repo

Milestones: prototype and tests (TM2) $\rightarrow$ estimators and first ECA curves $\rightarrow$ five-seed experiments and frozen figures $\rightarrow$ final demo, documentation and review (TM3/TM4).

Expected outcome and limitations

Outcome. A reproducible map of when prequential, requential and Bayesian model parts agree, and when an apparent epiplexity difference is caused by locality, optimisation, sampler or compute.

- A positive result: MLP/CNN task rankings reproduce the ECA reference with non-overlapping uncertainty intervals.
- A negative result is informative: it identifies observer dependence or an estimator failure.
- Small datasets, finite sweeps and imperfect loss floors limit absolute values; we therefore emphasise matched controls, curves in T, units and uncertainty.

Main question for discussion: which observer properties should count as part of the measured object?

References

M. Finzi, S. Qiu, Y. Jiang, P. Izmailov, J. Z. Kolter, A. G. Wilson. *From Entropy to Epiplexity: Rethinking Information for Computationally Bounded Intelligence*. arXiv:2601.03220, 2026.

S. Qiu, M. Finzi, Y. Zheng, K. Zhang, A. G. Wilson. *Requential Coding: Pushing the Limits of Model Compression with Self-Generated Training Data*. arXiv:2607.11883, 2026.

E. Donoway, H. Joren, F. Roger, J. Leike. *Excess Description Length of Learning Generalizable Predictors*. arXiv:2601.04728, 2026.